ct-01 — AtCoder Beginner Contest 475

Link	https://atcoder.jp/contests/abc475/tasks
Date attempted	2026-09-12 (live, A–B); C and D upsolved afterward — exact upsolve date not recorded
Problems	7 (A–G)
Solved in contest	2 — A, B
Solved after	2 — C, D
Unsolved	E (partial — upsolved with help, see `e.md`), F, G (not attempted — link-only stubs)
Concepts	segmented sieve, canonical coin systems, range covering on a line, binary search with the STL, prefix descent, binary trie, fenwick tree, offline query processing, complexity accounting, canonical representatives, fix two rows and sweep, sqrt from a product bound, divisor-count function, exchange argument, per-prime exponent caps, bounded candidate enumeration

1. The set

#	problem	verdict	key concept	one-line idea
A	mnclr	AC in contest	string building	insert `o` between every adjacent pair of characters in a length ≤10 string
B	Change	AC in contest	canonical coin systems	change per purchase is `(1e6 - A_i) % 1000`, decomposed digit-wise into 1/10/100-yen coins and summed across purchases
C	Walk the Line	AC after contest	range covering on a line	for every "go-left-first" boundary (and, symmetrically, every "go-right-first" boundary), binary-search the farthest reachable point in the other direction using prefix/suffix sums
D	Alphametic Prime	AC after contest	segmented sieve	segmented-sieve every `|S|`-digit number, then test candidates smallest-first for a character ↔ digit bijection with `S`
E	Quiz Competition: Qualifiers	partial	prefix descent	not attempted live; upsolved afterwards to a correct-but-TLE `partition_point` descent, then to two accepted solutions — **full write-up in `e.md`** (cq-006)
F	Rectangle Filling	not attempted	canonical representatives	count *tight* rectangles (every border line holds a white cell) + 1; fix the top and bottom rows, sweep the columns, transpose so the short side is squared
G	Has Many Divisors	not attempted	exchange argument on exponents	the smallest optimum has non-increasing exponents over 2, 3, 5, … except for one "dipped" prime; only 281,944 such numbers are ≤ 10^{18}, so enumerate them once and filter by `x % D != 0`

Solution files: `a.cpp`, `b.cpp`, `c.cpp`, `c2.cpp`, `d.cpp`, `e.cpp`, `e2.cpp`, `e3.cpp`, `f.cpp`, `g.cpp` in this directory. E is the one problem here with its own document, `e.md` — three solutions and eight concepts outgrew a table row.

A naming note worth recording: **C's actual, working solution lives in `c2.cpp`**, not `c.cpp`. `c.cpp` holds only a header comment enumerating three planned approaches ($O(n^2)$ brute force, $O(n \log n)$ binary search, $O(n)$ two-pointer) and is reserved for the $O(n)$ version later; `c2.cpp` is the $O(n \log n)$ approach that was implemented and is treated as C's solved entry here. `f.cpp` and `g.cpp` contain only a problem-link comment and contributed no verdict (both links read `contests/abc476/tasks/abc475_*` — the wrong contest in the path; the correct URLs are in the

table above). E has since acquired three: `e.cpp` (the correct-but-TLE descent, kept as the historical record), `e2.cpp` (binary trie), and `e3.cpp` (offline + Fenwick, the one to submit) — all three are dissected in `e.md`.

---

2. Key concepts across the set

2.1 Range covering on a line (prefix sums + binary search) — used in C

What it is: given `n` points on a line connected by weighted edges, a start point `S`, and a distance budget `L`, find the maximum number of distinct points reachable, where "reachable" means some walk from `S` (revisiting edges is allowed and costs distance each time) stays within total distance `L`.

How it works: the walk's optimal shape is always one of two forms — go to the left extreme `l` first, then to the right extreme `r` (retracing through `S`), or the mirror image. For a fixed interval `[l, r]` containing `S`:

- cost(left-first) = `2·dist(S, l) + dist(S, r)`
- cost(right-first) = `dist(S, l) + 2·dist(S, r)`

and the interval is reachable iff `min(cost(left-first), cost(right-first)) ≤ L`. Zig-zagging more than once never helps, since every extra direction change re-pays a segment already paid for without visiting anything new. Precomputing prefix sums `pre[i] = dist(town 0, town i)` and suffix sums `post[i] = dist(town i, town n-1)` turns `dist(i, j)` into an O(1) subtraction, and because all edge weights are positive, `pre` (and `post`) are strictly monotonic — which is exactly what makes binary-searching the farthest affordable endpoint valid.

Complexity: the implemented approach is O(n log n) — for every one of the `n` possible "first extreme" choices, binary search the farthest point reachable in the other direction. The O(n) refinement (reserved for `c.cpp`) replaces the binary search with a two-pointer sweep: as the first-extreme index moves monotonically outward, the farthest-reachable-in-the-other-direction index also moves monotonically, so it never needs to be re-searched from scratch.

How it showed up here: `leftCountFirst` fixes the left extreme `i` (looping `S-1` down to `0`, breaking as soon as `dist(S, i) > L`) and binary-searches the largest `curr` with `pre[curr] - pre[i] ≤ L - dist(S, i)`; `rightCountFirst` is the mirror image over `post`. The answer is `max` of the two functions' best results.

The recognisable signal: "single starting point, movement cost is the sum of edge weights traversed (revisits cost again), maximize the count of distinct nodes reached within a budget" is the shape that always decomposes into these two path orders. It shows up any time the graph is a line (or a tree, in a harder variant).

Implementation notes (from reviewing `c2.cpp`): the solution is correct, including a subtle point that's easy to get backwards — after moving to the first extreme, the code widens the result with `std::max(startPos, <binary-search result>)` rather than trusting the binary search alone. That guard matters: if the leftover budget after reaching `i` is too small to even re-pay the trip back to `S`, the binary search can return an index short of `S`, but `S` was already visited for free on the way out, so the true right boundary is never less than `S`. Two things worth tightening if this gets reused: the `if (left == minLeft) maxRight = startPos; else maxRight = std::max(startPos, left);` branch is redundant — `std::max(startPos, left)` alone produces the same value in both cases, since `minLeft < startPos` always holds inside `leftCountFirst` (mirrored in `rightCountFirst`); and the code mixes `size_t` (`pre.size()`), `ll`, and the sentinel `n` assigned into an `ll` — harmless at N ≤ 8000, but the signed/unsigned mix (`CONVENTIONS.md`'s "the `size_t` / `int` comparison" flag) is worth cleaning up before reusing this as a template for a larger-N problem. `std::midpoint` (used for the binary search's mid-point) is a C++20 addition specifically to avoid the classic `(low+high)/2` overflow bug.

Sources:

- USACO Guide — Two Pointers (the general technique the O(n) refinement targets)
- cppreference — std::midpoint (overflow-safe midpoint, used in the binary search)

See also §2.4 — the hand-rolled binary search described above rewritten with `std::upper_bound` / `std::lower_bound` / `std::partition_point`, which also shows that `post` is entirely redundant with `pre` and that the `std::max(startPos, ...)` guard discussed here is an artefact of the search window rather than a real edge case.

2.2 Segmented sieve — used in D

What it is: a way to enumerate primes in an arbitrary range `[L, R]` without allocating a sieve array of size `R` from `0`. It only needs the numbers up to `√R` (composite or not) to mark multiples inside the window `[L, R]`.

How it works: allocate a boolean array of size `R - L + 1`, representing candidacy for each number in `[L, R]`. For each `i` from `2` to `√R`, mark every multiple of `i` inside `[L, R]` as composite, starting from `max(i·i, first multiple of i that is ≥ L)`. Whatever survives is prime. Sieving multiples of every `i` (not just prime `i`) is slightly wasteful but still correct, since a composite `i`'s multiples are already covered by its prime factors — with `√R ≤ ~3163` here, the waste is immaterial.

Complexity: O((R − L + 1) log log R + √R) — dominated by the marking pass over the window.

How it showed up here: `S` has length 1–7, so the candidate primes are restricted to exactly `|S|`-digit numbers: `L = 10^(|S|-1)`, `R = 10^|S| - 1`. Worst case (`|S| = 7`) that's a window of ~9×10⁶ numbers — small enough that a segmented sieve over it, followed by an O(|S|) bijection check per candidate prime (smallest-first), comfortably fits the time limit.

The recognisable signal: whenever a problem needs "all primes in `[L, R]`" and `R` is too large to sieve from `0` but `R − L` (or `√R`) is small, that's a segmented sieve, not a full sieve.

Sources:

- CP-Algorithms — Sieve of Eratosthenes (§ Segmented Sieve / Find primes in range)

2.3 Canonical coin systems / digit decomposition — used in B

What it is: a coin system is *canonical* if the greedy algorithm (always take the largest denomination that fits) is provably optimal for every amount — not all systems are (e.g. some real-world denominations aren't).

How it works: a base-10 positional system with denominations `1, 10, 100, 1000, ...` is canonical, because ten of any denomination always equals exactly one of the next, so an optimal representation never uses ten or more of a smaller coin — which forces the optimal count of each denomination to be exactly that digit of the amount (own reasoning from the positional-notation definition, not stated explicitly on the source below, which only defines canonicity in general).

Complexity: O(1) per amount (or per digit, if you want the count broken out by denomination).

How it showed up here: each purchase's change is `(1000 - A_i % 1000) % 1000` — computed in the code as `(1_000_000 - A_i) % 1000` to dodge C++'s negative-modulo behavior, since `1e6` is a multiple of `1000`. Each purchase's change is independent of every other purchase's (Takahashi only ever pays with 1000-yen bills, never his accumulated coins), so the final coin counts are just the digit-wise sum, across all `N` purchases, of each purchase's change.

The recognisable signal: any "minimum coins for change" problem where the denominations are powers of a fixed base reduces to digit arithmetic — no DP needed, unlike change-making with arbitrary denominations.

Sources:

- Wikipedia — Change-making problem (§ Canonical coin systems)

2.4 Binary search with the STL — used in C

What it is: `<algorithm>`'s three binary-search primitives — `std::lower_bound`, `std::upper_bound`, `std::partition_point` — used in place of the hand-rolled loop in `c2.cpp`. All three answer the same question ("where does a monotone property flip?") and differ only in how the property is spelled: the first two compare each element against a fixed value, the third takes an arbitrary predicate.

How it works:

Step 1 — isolate the element. This is the enabling move, and it is algebra, not API knowledge. `lower_bound` / `upper_bound` never see the index, only the element, so everything that does not depend on the search variable must move to the right-hand side. `c2.cpp`'s predicate

```
pre[curr] - pre[minLeft] <= totalCostLocal      ->      pre[curr] <= pre[minLeft] + totalCostLocal
                                                       ^element^    ^-- constant for this iteration --^
```

If that isolation is awkward, or forming the right-hand constant risks overflow, that is precisely the signal to use `partition_point` instead.

Step 2 — pick the boundary. On an **ascending** range `v` and value `x`. Both functions return a *first*; the "last" forms are that minus one. Convert iterator to index with `- v.begin()`:

want	call
first `i` with `v[i] >= x`	`lower_bound(b, e, x)`
first `i` with `v[i] > x`	`upper_bound(b, e, x)`
last `i` with `v[i] < x`	`lower_bound(b, e, x) - 1`
last `i` with `v[i] <= x`	`upper_bound(b, e, x) - 1`

Both return `end()` when nothing qualifies and both can return `begin()`, so a `-1` is only safe when the first element is *provably* a match.

Step 3 — descending ranges need a comparator. A plain `lower_bound` on the non-increasing `post` array is **undefined behaviour**, not a wrong index: the range must be partitioned with respect to the comparison used. Either pass the order the range is actually sorted in — `std::lower_bound(b, e, x, std::greater<ll>{})`, which returns the first element *not greater than* `x` — or avoid the situation entirely (see "How it showed up here"). Argument order differs between the two and is a live trap: `lower_bound` calls `comp(*it, value)`, `upper_bound` calls `comp(value, *it)`.

The general form. `partition_point` returns the first element for which a predicate is **false**, given that the range is true-then-false throughout: *"all elements before `iter` satisfy `p`, while all elements starting from `iter` do not"*. So `pp` is the first false, `pp - 1` the last true, `pp == last` means all true, `pp == first` means all false. The law is that `p` must run **true-then-false, never false-then-true** — if the natural predicate reads backwards, negate it and the answer moves from `pp - 1` to `pp`. A non-monotone predicate is undefined behaviour, not a debuggable wrong answer. The other two are this function with the predicate baked in:

```
std::lower_bound(b, e, x)  ==  std::partition_point(b, e, [&](auto&& v){ return v <  x; });
std::upper_bound(b, e, x)  ==  std::partition_point(b, e, [&](auto&& v){ return !(x < v); });
```

which is also how to re-derive which is which: `lower_bound` partitions on `< x`, so its first-false is the first `>= x`.

Complexity: O(log n) comparisons — or O(log n) *applications of the predicate* for `partition_point`, so if the predicate is itself an O(n) feasibility check the total is O(n log n). All three are O(log n) only on **random-access** iterators; on a `std::set` or `std::list` the comparisons stay logarithmic but

the iterator increments are linear, so use the container's member `.lower_bound()` there. `std::views::iota` is random-access and safe.

How it showed up here: `c2.cpp`'s two hand-rolled loops both collapse. Three findings came out of the rewrite, in increasing order of transferability:

1. **`post` is entirely redundant with `pre`.** For `i <= j`, `post[i] - post[j] == pre[j] - pre[i]` — both are `dist(i, j)`. Dropping `post` means only one ascending array is ever searched, so the descending-range problem and `std::greater` never arise. The mirror loop becomes `lower_bound(pre.begin(), pre.begin() + s + 1, pre[s] - budget)`.
2. **The `if (left == minLeft) ... else std::max(startPos, left)` guard was an artefact of searching too wide a window**, not a real edge case. It existed because the search range `[minLeft, n)` included indices left of `S`, so the search could return a point short of `S` even though `S` was already visited for free on the way out. Starting the range at `pre.begin() + s` makes the result `>= s` by construction and the `-1` provably in-bounds: `pre[s] <= pre[s] + budget` always holds, so the window's first element is always a match. **Constraining `[first, last)` to the indices the problem permits is what removes the edge case** — most binary-search off-by-ones are really "I searched a wider window than the problem allows."
3. **The cost accounting has to be chosen once and kept.** The rewrite charges `2 * dist(S, extreme)` up front and then only `dist(S, far end)`, which makes the leftover budget a clean per-iteration constant. `c2.cpp` instead charges one trip out and then the full walk from the extreme — same total, but the two are *not* interchangeable mid-loop, and mixing them silently triple-charges `dist(S, extreme)`. (This was hit for real while drafting the `std::greater` variant for this section; a stress test against an O(n^2) brute force caught it.)

The resulting solution, verified against an O(n^2) brute force and against `c2.cpp` on 4,500 random tests including the `A_i = 10^9` / `L = 10^18` extremes:

```
ll best = 1;

// left extreme first, then as far right as the rest of the budget allows
for (ll i = s; i >= 0; --i) {
    ll spent = 2 * (pre[s] - pre[i]);        // out to i and back through S
    if (spent > l) break;                    // monotone in i -> safe to stop
    ll r = std::upper_bound(pre.begin() + s, pre.end(),
                            pre[s] + (l - spent)) - pre.begin() - 1;
    best = std::max(best, r - i + 1);
}

// mirror: right extreme first, then as far left as the rest allows
for (ll i = s; i < n; ++i) {
    ll spent = 2 * (pre[i] - pre[s]);
    if (spent > l) break;
    ll j = std::lower_bound(pre.begin(), pre.begin() + s + 1,
                            pre[s] - (l - spent)) - pre.begin();
    best = std::max(best, i - j + 1);
}
```

The `partition_point` form of the same two loops, which needs no algebra at all:

```
const ll budget = l - spent;

// loop A: cost grows with the index -> "affordable" is already true-then-false
auto affordable   = [&](ll p) { return p - pre[s] <= budget; };
ll r = std::partition_point(pre.begin() + s, pre.end(), affordable) - pre.begin() - 1;

// loop B: walking left, the affordable towns are the ones NEAREST S, i.e. on the
// right of the window -> "affordable" would be false-then-true, so negate it and
// the answer moves from (pp - 1) to pp itself.
auto unaffordable = [&](ll p) { return pre[s] - p > budget; };
ll j = std::partition_point(pre.begin(), pre.begin() + s + 1, unaffordable) - pre.begin();
```

Loop B's in-bounds proof mirrors loop A's: the window *ends* at index `s` and `unaffordable(pre[s])` is `0 > budget`, i.e. false, so there is always at least one false element and `pp` is never the window's `end`.

The recognisable signal: any monotone "largest/smallest index that still fits a budget" is one of these three calls. Reach for `lower_bound` / `upper_bound` when the predicate genuinely is "element vs. a constant" — it is shorter and needs no lambda. Reach for `partition_point` when the predicate involves anything else. In particular **"binary search the answer"** — *minimum capacity to ship in D days*, *maximise the minimum distance*, *smallest radius covering everything* — has no array to call `lower_bound` on and is always one `partition_point`, usually over an index range rather than over data:

```
auto days = std::views::iota(1LL, hi + 1);              // lazy, no allocation
ll ans = *std::ranges::partition_point(days, [&](ll d) { return !feasible(d); });
```

That form is the one that transfers; this problem's `upper_bound` is merely the tighter code for this problem.

Pitfalls worth a reflex:

- **Overflow moves to the right-hand side.** Isolating the element means building a constant like `pre[s] + (l - spent)`. Here `pre` maxes at `7999 * 10^9 ~ 8*10^12` against `L <= 10^18`, so the sum is ~`10^18` and safe in `long long` (max ~`9.2*10^18`) — but this is exactly what step 1 can break, so check it every time.
- **State why the predicate is monotone over *this* window** before writing the call. Here `pre` is strictly increasing because `A_i >= 1`; a zero weight would still be fine (non-decreasing), a negative one would invalidate the whole approach.
- **`pp == end()` is a normal case, not an error** — in loop A it means "everything is affordable" and the arithmetic handles it. Only guard it before dereferencing.
- Using `- v.begin()` yields a signed `ptrdiff_t`, which also clears the `size_t` / `ll` mixing flagged in §2.1.
- C++20 `std::ranges::lower_bound(pre, x)` takes the container directly and supports projections (`std::ranges::lower_bound(items, x, {}, &Item::weight)`). In `std::ranges` these are customization point *objects*, so unlike the classic function templates they can be bound to a variable and passed around.

Sources:

- cppreference — std::lower_bound — *"Iterator to the first element which is not ordered before `value`"*; comparator called as `comp(*iter, value)`
- cppreference — std::upper_bound — *"Iterator to the first element which is ordered after `value`"*; comparator called as `comp(value, *iter)`; *"If `ForwardIt` is not a `LegacyRandomAccessIterator`, the number of iterator increments is linear"*
- cppreference — std::partition_point — *"all elements before `iter` satisfy `p`, while all elements starting from `iter` do not"*; O(log N) applications of `p`

2.5 Prefix groups of fixed-length bitstrings — used in E

What it is: take a collection of equal-length bitstrings and fix a prefix `P`. The strings that start with `P` form a **prefix group**. Prefix groups nest: the group of `P` splits into exactly two children, the group of `P1` and the group of `P0`, and nothing else.

How it works: sorted lexicographically, every prefix group is a **contiguous block**, because the sort compares the first differing bit, so all strings agreeing on `P` sit together. Inside that block the two children are also contiguous and adjacent — in descending order, `P1` first, then `P0` — so splitting a group costs one split point. The consequence is structural: any process that starts from *everyone* and repeatedly keeps *one side* of the split at the next bit position is, at every moment, holding exactly one prefix group. A population that looked like it needed re-scanning at every step is really one path down a binary tree.

A micro-example on some other data: strings `111, 110, 101, 011, 010` (already descending). "Keep those with bit 0 = `0`" leaves the block `011, 010`; "then keep bit 1 = `1`" leaves the same block, now labelled as prefix `01`. Every filter was a split point, never a scan.

Complexity: one split is a binary search on a sorted array — `O(log N)` probes, each an `O(1)` bit read — or `O(1)` at a trie node that stores its count (§2.6).

How it showed up here: after `d` questions, E's undetermined participants are exactly one prefix group of the **correctness bitstrings** (`1` = answered question `d` correctly). A qualify step keeps the `0` side, an eliminate step keeps the `1` side, so the whole selection procedure is a `K`-step **prefix descent**. The proof, and the two ways of reading one participant's verdict off the descent, are in `e.md` §3.1.

The recognisable signal: a procedure that filters a population position by position (question by question, bit by bit), each time keeping one side. That is a walk down nested prefix groups, never a simulation.

Sources: the full treatment with citations is `e.md` §2.1.

2.6 Counting keys in a prefix group while the keys change: trie vs. offline ranks + Fenwick — used in E

What it is: the question "how many live keys have prefix `P`?" asked many times, while keys are being replaced (a query removes one key and inserts another). A sorted array answers it until the first change, and re-sorting is what kills you (§2.7). There are two standard repairs, and they fail in different ways.

How it works:

- **Binary trie — a structure that *is* the sorted order.** One node per prefix, and each node stores `cnt` = how many live keys pass through it. Insert and erase walk `K` levels adjusting `cnt`; "how many live keys have prefix `P`" is just `cnt` at `P`'s node. No search at all. The cost is memory: up to `N·K` nodes, scattered across the heap.
- **Offline ranks + Fenwick — freeze the universe, track occupancy.** If every change is known in advance (it is whenever the queries are all in the input and none depends on an earlier answer), collect **every key that will ever exist** — the initial ones plus one per query — then sort and deduplicate *once*. Ranks in that static array never move, so a prefix group is a fixed contiguous rank range forever, still found by binary search. Only occupancy changes, and a Fenwick tree over ranks handles that: point update to add or remove a key, prefix sums for "how many live keys in this rank range". Absent keys sit in the array contributing 0.

Complexity: trie — `O(K)` per update and per descent, memory `O(N·K)` nodes. Offline + Fenwick — `O((N+Q) log(N+Q))` setup, then `O(K log(N+Q))` per descent, memory `O(N+Q)` keys.

How it showed up here: `e2.cpp` is the trie, `e3.cpp` is offline + compression + Fenwick. **The one with the extra log factor was faster on the clock** — 0.14 s / 8.3 MB against 0.19 s / 74 MB — because its working set is small contiguous arrays that stay in cache while the trie chases pointers (`e.md` §3.7).

The recognisable signal: "I need the sorted order of a set that keeps changing." Either use a structure that *is* the order (trie, balanced BST), or — if the queries are offline — fix the universe of keys up front and track which ones are present.

Sources: `e.md` §2.4–§2.7 — trie, Fenwick tree, coordinate compression, offline processing, each with its own citations.

2.7 Complexity accounting: cost every line in the loop — used in E

What it is: the discipline of costing *every* statement inside the repeated body, not just the part you designed.

How it works: write out the query loop line by line, put a cost next to each line, multiply by the trip count, and take the maximum. The line that dominates is usually the boring one — a copy, a re-sort, a `std::string` built per iteration — because nobody was thinking about it when estimating.

How it showed up here: `e.cpp`'s own estimate, `O(QK log N)`, was exactly right about the descent and left out the two lines above it in the same loop: `auto S = C;` (a copy of `N` strings of length `K`, ~3×10^{11} in total) and `std::sort` (~4.5×10^{12} in total). Measured over 60 s against a 2 s limit. The fix was never to change the descent — it was to **get the sort out of the query loop**, which is what both §2.6 structures do.

The recognisable signal: a solution whose "interesting part" is fast and which still TLEs. Look at the lines you did not design.

Sources: `e.md` §2.3, and the line-by-line table in `e.md` §3.4.

2.8 Counting distinct outcomes: count canonical representatives, not operations — used in F

What it is: when a problem asks "how many **distinct results** can the operations produce", different operations routinely produce the same result, so counting operations overcounts. The fix is to choose, for each result, one **canonical** operation that produces it — a normal form — and count those instead. You need two properties:

1. **Every result has one:** any operation can be reduced to a canonical one that gives the same result.
2. **Distinct canonical operations give distinct results:** you can *recover* the canonical operation from the result alone. This is the half people skip, and it is the half that needs a proof.

How it works: the usual canonical choice is the **minimal** operation — shrink it while the result does not change, until nothing more can be removed. Injectivity then usually comes from "the minimal operation is exactly the bounding shape of what changed".

A micro-example on a different instance: a 1D row of `.` and `#`, and the operation "paint the interval `[l, r]` black". Painting `[l, r]` gives the same row as painting the interval shrunk to its first and last `.` inside it, so the canonical intervals are those whose **both endpoints are `.`**. Injectivity: the newly blackened cells are exactly the `.`s inside `[l, r]`, so their leftmost and rightmost positions are `l` and `r` — the result gives the interval back. With `c` dots in the row, the answer is `c(c+1)/2 + 1` (the `+1` is doing nothing). No simulation, no hashing of states.

Complexity: the reformulation costs nothing. What it buys is replacing "hash every reachable state" (cost proportional to the state size, per operation) with a pure counting problem over the canonical shapes.

How it showed up here: F is the 2D version. Filling rectangle `R` gives the same grid as filling `R` with its top row removed whenever that top row is already all black inside `R`, and likewise for the bottom row and the left and right columns. So the canonical rectangles are the **tight** ones: **the top row, bottom row, left column and right column of `R` each contain at least one white cell inside `R`**. Injectivity: the newly blackened cells are the white cells of `R`, each of `R`'s four border lines contains one of them, so their bounding box is exactly `R`. A tight rectangle also always changes at least one cell, so it never collides with "do nothing". **Answer = (number of tight rectangles) + 1.**

(Both editorials state this reformulation; the injectivity argument above is written out here as own reasoning.)

The recognisable signal: "count the number of distinct grids / strings / states obtainable", with an operation whose result depends only on the operation's footprint clipped to what actually changes.

Sources:

- AtCoder — ABC475 F official editorial (Japanese) — maps each state to its minimum bounding rectangle; counts rectangles whose four border lines each contain a white cell, plus 1
- AtCoder — ABC475 F user editorial by spheniscine (English)

2.9 Fix two rows, sweep the columns — and square the short side — used in F

What it is: the standard way to count sub-rectangles with a property. There are $O(H^2W^2)$ rectangles. Fixing the top row u and bottom row d ($O(H^2)$ pairs) reduces each pair to a **1D problem over the columns**, which you solve in $O(W)$ with a single sweep. Total $O(H^2W)$.

How it works:

- **Incremental column aggregates.** For a fixed u, as d moves down one row, any per-column summary of rows $u..d$ ("does this column contain a white cell in rows $u..d$?") updates in $O(W)$ by folding in row d. No 2D prefix sums needed.
- **Counting pairs (l, r) in one sweep.** Sweep the right column r. If the valid left ends for r are "eligible columns at or before some position $p(r)$", then their count is the **rank** of $p(r)$ among the eligible columns, which a running counter gives you for free as the sweep passes $p(r)$.
- **The square root hiding in a product bound.** A constraint like $H \cdot W \le 2\times10^5$, with no separate bound on H or W, means $\min(H, W) \le \sqrt{(2\times10^5)} \approx 447$. So **transpose if needed to make H the short side**, and then $H^2W = H\cdot(HW) \le 447 \times 2\times10^5 \approx 9\times10^7$, halved again because $d \ge u$. Without the transpose, $H = 2\times10^5, W = 1$ costs 2×10^{10}.

Complexity: $O(\min(H,W) \cdot H \cdot W) = O(HW\cdot\sqrt{(HW)})$.

How it showed up here: for a fixed (u, d), call a column **good** if it has a white cell somewhere in rows $u..d$. A tight rectangle's left and right columns must be good. Its top row must contain a white cell in $[l, r]$, which is the condition $l \le \text{lastWhite}_u(r)$ (the last white cell of row u at or before r), and likewise for row d. So for a good right column r, the valid l are exactly the good columns $\le \min(\text{lastWhite}_u(r), \text{lastWhite}_d(r))$. The white cell at $\text{lastWhite}_u(r)$ makes that column good itself, so "how many good columns are $\le$ it" is just the running good-counter at the moment the sweep passed it. The answer for r is $\min(kTop, kBot)$.

The recognisable signal: "count sub-rectangles such that…" together with a bound on the *product* $H\cdot W$. The product bound is the problem telling you which side to square.

Sources:

- AtCoder — ABC475 F user editorial by spheniscine (English) — transposes so `H ≤ W`, fixes top and bottom rows, maintains filled columns incrementally, `O(H²W)` = `O(HW√(HW))`
- AtCoder — ABC475 F official editorial (Japanese) — the transposed route: fix the left and right columns instead, with next-white-cell tables `f`, `g`; same `O(HW·min(H,W))`

2.10 The divisor-count function, and the exchange argument on exponents — used in G

What it is: if `n = p₁^e₁ · p₂^e₂ ⋯ p_k^e_k`, then the number of divisors is `d(n) = (e₁+1)(e₂+1)⋯(e_k+1)`. It depends **only on the multiset of exponents**, not on which primes carry them.

How it works: because the primes don't matter to `d(n)` but matter a lot to `n`, the smallest number with a given exponent multiset puts the **largest exponents on the smallest primes**. The proof is an exchange: if primes `q < r` have exponents `e_q < e_r`, swap them. `d(n)` is unchanged, and `n` is multiplied by `(q/r)^(e_r − e_q) < 1`. So a number that is minimal for its divisor count has **non-increasing exponents over consecutive primes 2, 3, 5, 7, …** with no gaps (a gap is exponent 0 followed by a positive one, which the same swap fixes). These are the **highly composite**-style numbers.

Micro-example: `12 = 2²·3` and `18 = 2·3²` both have 6 divisors. The exchange turns `18` into `12`.

Complexity: once you search exponent vectors instead of numbers, the search space is tiny. Up to `10¹⁸` there are at most 15 distinct primes (the product `2·3·5·…·47 ≈ 6.1×10¹⁷` fits, and multiplying in 53 overflows `10¹⁸`), and exponents are at most 59.

The recognisable signal: "maximise the number of divisors of `x ≤ N`" with `N` far too large to scan. Search over exponent vectors on the smallest primes, never over `x`.

Sources:

- CP-Algorithms — Number of divisors / sum of divisors — `d(n) = (e₁+1)·(e₂+1)⋯(e_k+1)`; `d(a·b) = d(a)·d(b)` for coprime `a, b`
- Wikipedia — Highly composite number — "the sequence of exponents must be non-increasing"

2.11 "Not a multiple of D" is a cap on one prime's exponent — used in G

What it is: `D | x` holds iff `v_p(x) ≥ v_p(D)` for **every** prime `p`, where `v_p` is the exponent of `p`. Negating it: `x` is **not** a multiple of `D` iff **some** prime `p | D` has `v_p(x) < v_p(D)`. An existential with a single witness.

How it works: the useful move is to fix the witness `p` in the *proof* and leave every other prime alone. Only `p`'s exponent is constrained, so the exchange argument of §2.10 still applies to every pair of primes not involving `p`. It also applies to a prime `q < p` against `p` itself: if `v_q(x) < v_p(x)`, swapping them lowers `v_p` (so `p` stays a valid witness) and shrinks `x`. What survives is:

exponents non-increasing over the primes, except that the witness prime may dip below the ones after it.

Micro-example: "not a multiple of $12 = 2^2 \cdot 3$" means $v_2(x) \le 1$ **or** $v_3(x) = 0$. $x = 10 = 2 \cdot 5$ uses the first cap, $x = 8 = 2^3$ the second.

Complexity: free as a statement — and note what it avoids. D can be up to 10^{18} and **never needs to be factorised**: the witness only exists in the proof, and the actual test is $x \% D != 0$. If the witness prime is huge (say D is a prime near 10^{18}), it sits beyond every prime the candidate uses, $v_p(x) = 0$, and the "dip" is invisible. The candidate is then an ordinary non-increasing one, which the enumeration already contains.

How it showed up here: it turns G's constraint from "avoid a set of numbers" into "allow one exception to the ordering", which is what makes a D-independent enumeration possible (§2.12).

The recognisable signal: "not divisible by D", "$\gcd$ constraint", or "does not contain factor …" alongside a problem that is otherwise about prime exponents.

Sources: own reasoning from the definition of divisibility via prime exponents; the resulting structure (non-increasing except the witness prime) is the one stated in AtCoder — ABC475 G official editorial (Japanese) and the English user editorial by Xterfusion.

2.12 Enumerate a small structured candidate set once, filter per query — used in G

What it is: when a proof shows the answer always lies in a structured family, and that family turns out to be small when you actually **count** it, generate the whole family once by DFS and answer each query with a linear scan.

How it works: DFS over the primes in order, carrying the current value, the bound on the next exponent, a flag for "the one allowed dip has been used", and the running divisor count $\prod(e+1)$. Record the value at every node (it means "all remaining exponents are 0"). Prune as soon as multiplying by the next prime would exceed the limit. Then sort by value and deduplicate. The same number can be reached by more than one path, for example "dip to 0 here" versus "end here", so the dedupe is required.

Overflow is the one sharp edge. $10^{18} \times 53$ exceeds $\texttt{unsigned long long}$'s ~$1.8 \times 10^{19}$, so the pruning test must be written as $v > LIM / p$ (or done in $\texttt{__int128}$) *before* multiplying, never as $v * p > LIM$ after.

Complexity: the family below 10^{18} has **281,944** members. That is the editorial's count, and the DFS below reproduces it exactly, which doubles as a check on the pruning. Per query it is one scan, so $T \le 10$ costs ~3×10^6 operations, and none of it depends on D.

The recognisable signal: $N \le 10^{18}$ with a tiny T and an **extremal-number** objective (most divisors, smallest with property X). Prove a structure theorem, count the family, and if it's under ~10^7, enumerate it.

Sources:

- AtCoder — ABC475 G official editorial (Japanese) — "there are only 281944 integers $x \le 10^{18}$ satisfying the above conditions"

---

3. From brute force to optimal — E, F, G

A–D were solved and are reviewed in §2.1–§2.4. This section covers only the three unsolved problems. Every algorithm with code below was compiled and checked against the official samples and a brute force before being written down. F was stress-tested against an exhaustive "apply every rectangle, hash the grid" oracle on 1,500 random grids up to 5×5. G was checked against a divisor-count sieve on 3,000 random `(N, D)` pairs with `N ≤ 2×10⁵`, comparing divisor counts rather than values, since any optimum is accepted. Both showed 0 mismatches.

E — Quiz Competition: Qualifiers

The full treatment, with all three solutions and their code, lives in `e.md` §3. This is the ladder in outline.

- **Rung 1 — simulate the statement.** For each query, walk the `K` questions and scan all `N` participants at each one: `O(NK)` per query, `Q·N·K = 3×10¹¹`. Correct and hopeless. This is also the brute force every later rung was checked against.
- **Rung 2 — see the prefix group (§2.5).** Switch to correctness bitstrings, and the undetermined set becomes one prefix group, so the procedure is a `K`-step descent with one split per step. On a descending-sorted array each split is a `partition_point`: `O(K log N)` per query. **This is `e.cpp`**, and its descent is correct (0 mismatches over 8.9 million exhaustive evaluations). It still TLEs because it copies and re-sorts all `N` strings inside every query (§2.7): `O(NK log N)` per query, measured over 60 s.
- **Rung 3a — a structure that is the sorted order (§2.6).** A binary trie with a count per node. A flip is erase, toggle, insert, and the descent reads one count per level with no search. `O((N+Q)·K)`. **`e2.cpp`**: 0.19 s / 74 MB.
- **Rung 3b — freeze the universe offline (§2.6).** Collect every key any participant will ever hold, sort and deduplicate once, and keep a Fenwick tree of live counts over the ranks. The descent keeps `partition_point` exactly as `e.cpp` wrote it, now over a static array. `O(QK log(N+Q))`. **`e3.cpp`**: 0.14 s / 8.3 MB — **the one to submit.**

What to take from the ladder: rung 2 is the insight and rung 3 is data-structure plumbing. The rung-2 → rung-3 gap was never about the algorithm. It was about a sort sitting inside a loop. The four silent bugs hit on the way are catalogued in `e.md` §3.10.

F — Rectangle Filling

- **Rung 1 — apply every rectangle, hash the grid.** Enumerate all `H(H+1)/2 · W(W+1)/2` rectangles, paint each onto a copy, and insert it into a `std::set`. The count of rectangles alone reaches ~10^{10} at `H = W ≈ 447`, and each state costs `O(HW)` to build and compare. Only useful as an oracle, which is exactly how it was used here.
- **Rung 2 — count canonical rectangles instead of states (§2.8).** Answer = number of **tight** rectangles (each of the four border lines contains a white cell inside `R`) + 1. Check each rectangle in `O(1)` with per-row and per-column prefix counts of white cells: `O(H²W²)` ≈ 10^{10}. The hashing is gone and the answer is now a *count*, but it is still quadratic in the number of rows *and* columns.
- **Rung 3 — fix `(u, d)`, sweep the columns, square the short side (§2.9).** For each top row `u`, grow `d` downward, maintaining `good[c]` = "column `c` has a white cell in rows `u..d`". Then one sweep over the columns adds, for each good right column, the number of good columns at or before `min(lastWhite_u, lastWhite_d)`. Transpose first so `H ≤ 447`. `O(H²W) ≤ ~4.5×10⁷` after the `d ≥ u` halving.

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int h, w;
    std::cin >> h >> w;
    std::vector<std::string> g(h);
    for (auto& row : g) std::cin >> row;

    if (h > w) {                                    // square the SHORT side: h <= 447
        std::vector<std::string> t(w, std::string(h, '.'));
        for (int r = 0; r < h; ++r)
            for (int c = 0; c < w; ++c) t[c][r] = g[r][c];
        g = std::move(t);
        std::swap(h, w);
    }

    long long ans = 1;                              // the "do nothing" state
    std::vector<char> good(w);                      // column has a '.' in rows u..d
    for (int u = 0; u < h; ++u) {
        std::fill(good.begin(), good.end(), 0);
        for (int d = u; d < h; ++d) {
            for (int c = 0; c < w; ++c) good[c] |= (g[d][c] == '.');

            // sweep the right column r; kTop / kBot = number of good columns
            // at or left of the last '.' seen so far in row u / row d
            long long goodSoFar = 0, kTop = 0, kBot = 0;
            for (int c = 0; c < w; ++c) {
                if (good[c]) ++goodSoFar;
                if (g[u][c] == '.') kTop = goodSoFar;
                if (g[d][c] == '.') kBot = goodSoFar;
                if (good[c]) ans += std::min(kTop, kBot);
            }
        }
    }
    std::cout << ans << '\n';
}
```

Three details worth a second look:

- **`kTop` and `kBot` start at 0**, so while row `u` (or `d`) has no white cell at or before `r`, the right column contributes nothing. That is the "top row must contain a white cell" condition enforced for free.
- **`u == d` needs no special case.** The top and bottom row are the same row, `kTop == kBot`, and the count is exactly the 1D problem from §2.8.
- **The answer overflows `int`.** An all-white 447×447 grid gives `10,025,616,385` and an all-white 1×200,000 grid gives `20,000,100,001` (`W(W+1)/2 + 1`). Both were measured, and they are the reason `ans` and the counters are `long long`.

Measured: 0.06 s on the all-white 447×447 grid, 0.06 s on a random 400×500, and 0.00 s on 1×200,000, against a 2 s limit.

G — Has Many Divisors

- **Rung 1 — scan every `x ≤ N`.** Count divisors by trial division and keep the best non-multiple of `D`: `O(N√N)`. Fine for the tiny samples, not even startable at `N = 10¹⁸`. With a divisor-count sieve it becomes `O(N log N)` up to ~10^7, and that version is the oracle used for verification here.
- **Rung 2 — ignore `D` and look at what a winner looks like (§2.10).** Only exponents matter to `d(x)`, so a smallest-optimal `x` has non-increasing exponents over 2, 3, 5, …. Without `D`, the answer is a highly composite-style number, and there are few enough of them to enumerate.
- **Rung 3 — put `D` back as one exception (§2.11).** "Not a multiple of `D`" means some prime `p | D` has `v_p(x) < v_p(D)`. Take the **smallest** optimal `x` and fix that witness `p`. The exchange argument still forces non-increasing exponents on every other prime, and forces `v_q(x) ≥ v_p(x)` for `q < p`. So **the exponent sequence is non-increasing except that one prime may dip below the primes after it.**
- **Rung 4 — enumerate that family once, filter per query (§2.12).** DFS over the first 16 primes (up to 53, because a prime dipped to 0 can push the 15th used prime one step further), allowing at most one dip. That gives 281,944 distinct candidates ≤ 10^{18}. For each test, scan them in increasing order up to `N` and keep the one with the most divisors that satisfies `x % D != 0`. `D` never needs factorising.

```cpp
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using u64 = unsigned long long;

constexpr u64 LIM = 1'000'000'000'000'000'000ULL;
constexpr int P[16] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53};

std::vector<std::pair<u64, u64>> cand;            // (value, divisor count)

// bound  : max exponent allowed for the next "normal" prime
// dipped : has the one exponent allowed to break the order been used yet
void dfs(int i, u64 val, int bound, bool dipped, u64 divs) {
    cand.push_back({val, divs});                  // all remaining exponents = 0
    if (i == 16) return;
    if (!dipped) dfs(i + 1, val, bound, true, divs);          // dip to exponent 0
    u64 v = val;
    for (int e = 1; e <= bound; ++e) {
        if (v > LIM / P[i]) break;                // v * P[i] would exceed 1e18
        v *= P[i];
        dfs(i + 1, v, e, dipped, divs * (e + 1));             // normal: e is the new bound
        if (!dipped && e < bound)
            dfs(i + 1, v, bound, true, divs * (e + 1));       // dip: bound unchanged
    }
}

int main() {
    dfs(0, 1, 60, false, 1);
    std::sort(cand.begin(), cand.end());
    cand.erase(std::unique(cand.begin(), cand.end()), cand.end());   // 281,944 left

    int t;
    std::cin >> t;
    while (t--) {
        u64 n, d;
        std::cin >> n >> d;
        u64 best = 0, ans = 0;
        for (auto [x, divs] : cand) {
            if (x > n) break;                     // sorted by value
            if (x % d != 0 && divs > best) { best = divs; ans = x; }
        }
        std::cout << ans << '\n';
    }
}
```

Things worth noticing:

- **The candidate set is a superset, and that's fine.** The DFS allows a dip at any prime, not only at primes of `D`, and doesn't require the dip to stay below `v_p(D)`. Extra candidates only cost scan time. The `x % D != 0` filter plus "most divisors" picks the right one, because the proof guarantees the smallest optimum is somewhere in the set.
- **Any optimum is accepted.** On sample 2 (`N = 17, D = 4`) this prints `6` where the official output is `15`. Both have 4 divisors, and the statement says "output any one of them". Compare divisor counts, not values, when checking against the samples.
- **`dfs(0, 1, 60, …)`** — the starting bound 60 is simply "larger than any exponent that fits", since $2^{60} > 10^{18}$. The overflow guard stops the loop long before it matters.

Measured: 0.04 s for 10 tests at $N = 10^{18}$, candidate generation included.

---

4. Key ideas that would have helped

- **E's TLE was in the line nobody was looking at.** The descent was `O(K log N)` and correct. The `std::sort` above it in the same loop was `O(NK log N)` per query. With `N·Q ≈ 1.5×10⁹` stated right there in the constraints, anything touching all `N` participants per query was ruled out before any cleverness began. Cost the whole loop body (§2.7).
- **F's "distinct states" wording is the whole problem.** Counting operations overcounts, and hashing states is far too slow. The move is to name a canonical operation per state (the tight rectangle) and count those (§2.8). After that it is a standard "count sub-rectangles" sweep.
- **F's `H·W ≤ 2×10⁵` is a square root in disguise.** It says `min(H, W) ≤ 447`, so squaring the short side is affordable and squaring the long side is not. Transposing is one line and decides whether the solution passes.
- **G's `N ≤ 10¹⁸, T ≤ 10` with "most divisors" means "search exponent vectors".** Then negate the constraint: "not a multiple of `D`" is "one prime's exponent capped", which is only one exception to the highly composite ordering (§2.11). `D` never needs factorising.
- **F's answer overflows `int`** (~10^{10} on an all-white grid). A count of rectangles is `0((HW)²)` in the worst case, so check it against `2³¹` before choosing the type.
- **C's `N ≤ 8000` was a green light to bank an AC before optimizing.** `8000² = 6.4×10⁷` — the brute-force $O(n^2)$ approach noted as "approach 1" in `c.cpp`'s header comment would have passed comfortably inside the time limit. Going straight for O(n log n) (or the O(n) two-pointer target) is the right instinct for practice, but on the clock, submitting the $O(n^2)$ version first and refining afterward banks the points without the risk of debugging the trickier binary-search edge case (the `std::max(startPos, ...)` guard covered in §2.1) live.
- **D's `|S| ≤ 7` bounded the whole search space to ≤ $9×10^6$ numbers.** The constraint doesn't need a number-theoretic shortcut — it's small enough that "sieve the window, then brute-force-check each candidate" is the intended complexity, and recognizing that early (rather than looking for a smarter construction) is what unlocks the problem.
- **`L ≤ 10^18` in C is itself a signal**, independent of the algorithm: it rules out `int` for any distance/cost variable well before you even design the approach. The submitted code got this right throughout (`ll` everywhere), which is worth keeping as a reflex — check the largest bound in the statement before writing the first line.

5. Reading the constraints

#	limits	what that was signalling
A	`|S|` ≤ 10	trivially small — any correct construction works, no complexity target
B	`N ≤ 1000`, `A_i ≤ 10^5`	O(N) simulation; values small enough for `int`, though the accumulated coin counts still fit comfortably
C	`N ≤ 8000`, `L ≤ 10^18`	`N` small enough for $O(n^2)$ or O(n log n); `L`'s huge bound demands `long long` / `int64_t` for every distance and budget variable
D	`|S|` 1–7	candidate search space capped at ~9×10^6 — segmented sieve + brute-force bijection check, not a closed-form construction
E	`1 ≤ M ≤ N ≤ 3×10^4`, `K ≤ 200`, `Q ≤ 5×10^4`	`K·Q = 10^7` is fine, but `N·Q ≈ 1.5×10^9` is not — each query needs to avoid re-simulating all `N` participants from scratch
F	`H·W ≤ 2×10^5`, no separate bound on `H` or `W`	a product bound means `min(H, W) ≤ √(2×10^5) ≈ 447`: transpose so the short side is the one squared, giving `O(HW·min(H,W)) ≈ 9×10^7`. The number of rectangles (~10^10) rules out enumerating them, and it also means the *answer* needs `long long`
G	`2 ≤ D ≤ N ≤ 10^18`, `T ≤ 10`	far too large to scan `x`, but a tiny `T` allows a per-test scan over a precomputed family. Search prime-exponent vectors (at most 15 distinct primes, exponents ≤ 59 below 10^18), not numbers. The 281,944-candidate family is small enough to enumerate outright

6. Post-mortem

Where the time went: not recorded.

What to drill:

- The segmented-sieve pattern from D (§2.2) — it's a clean, reusable primitive worth having ready without re-deriving the `max(i·i, first multiple ≥ L)` bound each time.
- ~~Replacing the hand-rolled binary search in `c2.cpp` with `std::upper_bound` / `std::lower_bound` directly on the `pre` / `post` arrays.~~ **Done 2026-09-16 — written up as §2.4.** Three things carried forward: `post` is redundant with `pre` (`post[i] - post[j] == pre[j] - pre[i]`), so only one ascending array ever needs searching; the `left == minLeft` branch was guarding a symptom of too wide a search window, not a real edge case; and `std::partition_point` — not `lower_bound` — is the form that generalises to "binary search the answer". `c2.cpp` itself is left untouched as the historical record.
- Still open from this line of work: the O(n) two-pointer version of C reserved for `c.cpp`. §2.4 removes the binary search's sharp edges but does not change the complexity class.
- A quick habit check: given a fresh problem, name the complexity class the constraints demand *before* picking an approach — C's `N ≤ 8000` supporting $O(n^2)$ is the concrete example from this contest.
- **The canonical-representative proof from F (§2.8)**, both halves: every outcome reduces to a canonical operation, *and* the canonical operation can be read back from the outcome. The 1D interval version (`c(c+1)/2 + 1`) is a two-minute warm-up that makes the 2D one mechanical.
- **The exchange argument on prime exponents from G (§2.10–§2.11)**, including how one existential constraint ("some prime is capped") leaves the rest of the exchange intact. It's the same proof shape as ct-02 D's exchange argument (§2.2 there), applied to exponents instead of prices.

Open: E, F and G are still unsolved in code — `f.cpp` and `g.cpp` are link-only stubs, and E's accepted solutions were written with help (cq-006).

- **E** is fully written up in `e.md`. Nothing is left to learn from it except re-deriving the prefix-group insight cold, which is worth doing as a timed exercise.
- **F** and **G** each have a verified route in §3. F is the better one to write first: the only new idea is §2.8, and the sweep is ABC-level implementation. G needs the proof in §2.11 more than it needs code.